Name : K .Dinesh

Hall Ticket No: 2303A52288

Batch No : 42

Task 1 – Problem Identification & AI-Assisted Ideation

Task:

Identify and refine a real-world problem using AI.

Prompt:

Suggest 3 real-world civic problems that can be digitized such as complaint systems, parking, or waste management.

Refine the selected problem into a clear problem statement and generate key features, target users, and expected impact.

Output:

1. Three real-world civic problems that can be digitized:
2. Citizen Complaint and Grievance Tracking
   Residents report potholes, streetlight failures, water leakage, illegal dumping, or noise issues, but follow-up is slow and opaque.
3. Smart Parking Availability and Enforcement
   Drivers waste time searching for parking, leading to congestion, pollution, and frequent rule violations.
4. Waste Collection Monitoring and Missed Pickup Resolution
   Households and businesses face irregular collection, overflowing bins, and no reliable way to track complaints or service quality.
5. Refined problem statement for one selected problem:
   Selected problem: Citizen Complaint and Grievance Tracking

Selected problem

1. Citizen Complaint and Grievance Tracking
   Residents report potholes, streetlight failures, water leakage, illegal dumping, or noise issues, but follow-up is slow and opaque.

Refined Problem Statement

Traditional complaint handling systems used by city councils are manual, slow, and often non-transparent. Citizens cannot reliably track complaint progress, while authorities struggle with delays, duplication, and poor coordination. A centralized digital complaint management system is needed to simplify complaint registration, enable real-time tracking, and improve resolution efficiency and accountability.

Target Users

1. Citizens: Register complaints and track status updates.
2. City Council Staff: Review, assign, and resolve complaints.
3. Administrators: Monitor performance, escalations, and service quality.

Key Features

1. Smart Complaint Registration
   Users can submit complaints with category, location, description, and photo proof through web/mobile.
2. Real-Time Tracking and Notifications
   Citizens receive status updates (Submitted, In Progress, Resolved) via dashboard, SMS, or email.
3. Admin Dashboard with Escalation Controls
   Officials can assign complaints to departments, enforce SLA deadlines, escalate overdue cases, and view analytics.

Task 2 – System Planning & Architecture Design

Task:

Design the system architecture with AI support.

Prompt used

Design a system architecture for a digital complaint management system. Include frontend, backend, database, data flow, API design, and database schema.

System Components

1. Frontend: React
   Used by citizens to submit and track complaints, and by admins to manage cases.
2. Backend: Node.js + Express
   Handles authentication, validation, complaint routing, status updates, and API responses.
3. Database: MongoDB
   Stores users, complaints, status history, and metadata.

Architecture Description

1. Presentation Layer
   Web or mobile interface where users submit complaints and view progress.
2. Application Layer
   Backend services process requests, enforce business rules, and trigger notifications/escalations.
3. Data Layer
   MongoDB persists user profiles, complaint records, and lifecycle states.

Data Flow

User -> Frontend -> API Request -> Backend -> Database -> Backend -> Frontend -> User

Flow Explanation:

1. User submits a complaint from the interface.
2. Frontend sends request to backend API.
3. Backend validates input and stores complaint in database.
4. Backend returns success/status response.
5. Frontend displays complaint ID and latest status to user.

Database Schema (Refined)

1. Users Collection

- user_id
- name
- email
- role (citizen, staff, admin)
- created_at

2. Complaints Collection

- complaint_id
- title
- description
- category
- status (Pending, In Progress, Resolved, Rejected)
- priority (Low, Medium, High)
- user_id (reference to Users)
- assigned_to (staff/admin reference)
- location
- created_at
- updated_at

LIBRARY DATABASE DESIGN AND SQL QUERIES

- user_id (reference to Users)
- assigned_to (staff/admin reference)
- location
- created_at
- updated_at

APIs / Modules

1. POST /complaints
   Create complaint
2. GET /complaints
   List complaints with filters
3. GET /complaints/:id
   Get complaint details
4. PUT /complaints/:id/status
   Update complaint status
5. DELETE /complaints/:id
   Remove complaint (admin-only)
6. GET /users/:id/complaints
   Get complaints submitted by a specific user

Workflow

1. Citizen submits complaint.
2. Backend validates and saves complaint.
3. Complaint appears in admin dashboard.
4. Admin assigns and updates status.
5. Citizen tracks updates until resolution.
6. Feedback can be collected after closure.

Expected Output Justification

1. Scalable architecture separates UI, logic, and data.
2. API-first design enables web and mobile clients.
3. Schema supports traceability, filtering, and reporting.
4. Role-based workflow improves accountability and resolution speed.

Task 3 – Modular Code Development with AI

Task:

Develop core modules of the application using AI.

Prompt:

Generate modular JavaScript code for a complaint management system including input handling, core logic,

and data storage. Include validation, error handling, and clean function-based structure.

Code

```javascript
// Data Storage Module
let complaints = [];

// Input Handling + Validation
function validateInput(title, description) {
  if (!title || !description) {
    throw new Error("Title and Description are required");
  }
  if (typeof title !== "string" || typeof description !== "string") {
    throw new Error("Invalid data type");
  }
}

// Core Logic Module
function addComplaint(title, description) {
  try {
    validateInput(title, description);

    const complaint = {
      id: complaints.length + 1,
      title: title.trim(),
      description: description.trim(),
      status: "Pending",
      created_at: new Date().toISOString()
    };

    complaints.push(complaint);
    return { success: true, data: complaint };

  } catch (error) {
    return { success: false, message: error.message };
  }
}

// Fetch All Complaints
function getComplaints() {
  return complaints;
}

// Update Complaint Status
function updateComplaint(id, status) {
  try {
    const complaint = complaints.find(c => c.id === id);

    if (!complaint) {
      throw new Error("Complaint not found");
    }

    complaint.status = status;
    return { success: true, data: complaint };

  } catch (error) {
    return { success: false, message: error.message };
  }
}
```

Input1

```
addComplaint("Garbage Issue", "Waste not collected in area")
```

Output1:


```
{
  "success": true,
  "data": {
    "id": 1,
    "title": "Garbage Issue",
    "description": "Waste not collected in area",
    "status": "Resolved",
    "created_at": "2026-04-09T13:11:24.599Z"
  }
}
```


Input2

```
addComplaint("", "")
```

Output2:

```
{
  "success": false,
  "message": "Title and Description are required"
}
```

Input3:

```
updateComplaint(1, "Resolved")
```

Output3:

```
{
  "success": true,
  "data": {
    "id": 1,
    "status": "Resolved"
  }
}
```

Task 4 – AI-Assisted Testing, Debugging & Optimization

Task:

Validate and improve your application.

Prompt used

Generate test cases, identify bugs, and suggest optimizations for a JavaScript complaint management system with input validation and in-memory storage.

Test Case 1 – Valid Input

```
createComplaint("Garbage Issue", "Not collected")
```

Output1

```
Success: Complaint added with status "Pending"
```

Test case 2-Empty input

```
createComplaint("", "")
```

Output2:

```
Error: "All fields are required"
```

Test Case 3 – Invalid Update

```
updateStatus(9999, "Resolved")
```

Output3:

```
Error: "Complaint not found"
```

Bugs Identified

No persistent storage (data lost after refresh)

ID generation using `Date.now()` may conflict in rare cases

No validation for status values

Fixes & Improvements

Added status validation

Improved ID generation logic

Structured error handling

Optimized Code (Improved Version)

```javascript
// Allowed status values
const VALID_STATUS = ["Pending", "In Progress", "Resolved"];

// Improved ID generator
let idCounter = 1;

// Data Store
const ComplaintStore = {
  complaints: [],

  add(complaint) {
    this.complaints.push(complaint);
  },

  getAll() {
    return this.complaints;
  },

  findById(id) {
    return this.complaints.find(c => c.id === id);
  }
};

// Validation
function validateInput(title, description) {
  if (!title || !description) {
    throw new Error("All fields are required");
  }
}

// Create
function createComplaint(title, description) {
  try {
    validateInput(title, description);

    const complaint = {
```

```javascript
function createComplaint(title, description) {
    const complaint = {
      title: title.trim(),
      description: description.trim(),
      status: "Pending"
    };

    ComplaintStore.add(complaint);
    return { success: true, data: complaint };

  } catch (err) {
    return { success: false, message: err.message };
  }
}

// Update
function updateStatus(id, status) {
  try {
    if (!VALID_STATUS.includes(status)) {
      throw new Error("Invalid status value");
    }

    const complaint = ComplaintStore.findById(id);

    if (!complaint) {
      throw new Error("Complaint not found");
    }

    complaint.status = status;
    return { success: true, data: complaint };

  } catch (err) {
    return { success: false, message: err.message };
  }
}
```

Task 5 – Documentation, Version Control & Reflection

Task:

Prepare the project for submission and collaboration.

Prompt used :

Generate a README file, add code comments, suggest Git commit messages, and write a reflection for a complaint management system project.

README (ai generated)

```markdown
# Digital Complaint Management System

## Setup
1. Install Node.js
2. Run the JavaScript file using node

## Usage
- Add complaints using createComplaint()
- View complaints using getComplaints()
- Update status using updateStatus()

## Features
- Complaint tracking
- Status updates
- Input validation
```

Code Documentation

```javascript
// Function to create a new complaint
// Validates input and stores complaint in memory
function createComplaint(title, description) {
  ...
}
```

Sample Commit Messages

Initial project setup

Added complaint creation module

Implemented input validation

Fixed status update bug

Optimized code structure

Reflection Report

->how ai helped

Generated initial code quickly

Assisted in debugging and testing

Helped design system architecture

->LImitation faced

Generated initial code quickly

Assisted in debugging and testing

Helped design system architecture

->Improvements Possible

Add database (MongoDB)

Build frontend UI

Deploy as full-stack app

Add authentication system